

Intro to REST

 <https://github.com/learn-co-curriculum/python-p4-intro-to-rest>  <https://github.com/learn-co-curriculum/python-p4-intro-to-rest/issues/new>

Learning Goals

- Build RESTful APIs that are easy to navigate and use in applications.

Key Vocab

- **Representational State Transfer (REST)**: a convention for developing applications that use HTTP in a consistent, human-readable, machine-readable way.
- **Application Programming Interface (API)**: a software application that allows two or more software applications to communicate with one another. Can be standalone or incorporated into a larger product.
- **HTTP Request Method**: assets of HTTP requests that tell the server which actions the client is attempting to perform on the located resource.
- **GET** : the most common HTTP request method. Signifies that the client is attempting to view the located resource.
- **POST** : the second most common HTTP request method. Signifies that the client is attempting to submit a form to create a new resource.
- **PATCH** : an HTTP request method that signifies that the client is attempting to update a resource with new information.
- **PUT** : an HTTP request method that signifies that the client is attempting to update a resource with new information contained in a complete record.
- **DELETE** : an HTTP request method that signifies that the client is attempting to delete a resource.

Introduction

In 2000, Roy Fielding was frustrated by the haphazard ways in which web applications were using HTTP. Specifically, he was frustrated with how URLs and their corresponding HTTP verbs were used differently for every single application. So, in his [Ph.D. dissertation](#) 

(https://www.ics.uci.edu/%7Efielding/pubs/dissertation/fielding_dissertation.pdf), he came up with **RESTn** (**RE**presentational **S**tate **T**ransfer) as a standardized way for web applications to structure their URLs.

Fielding also noticed the rise in web applications communicating with each other ([What is an API?](https://www.mulesoft.com/resources/api/what-is-an-api)  <https://www.mulesoft.com/resources/api/what-is-an-api>). He hoped that inter-application communication would get much easier if there was a standard way of forming URLs to access resources.

If you have been building applications for a while, there is a good chance that you have already worked with RESTful APIs. `json-server` follows REST conventions very strongly. If your application posts to Twitter, pulls in a feed of images from Instagram, or calls a list of locations from Google Maps, you are using a RESTful API to communicate between applications.

► *What is REST?*

Example REST Workflow

For a real world case study, let us pretend that you have a newsletter application. The following is a high-level view of how such an app might work:

1. You fill out the form on the 'New Newsletter' page and click submit.
 2. Data concerning you as the author, your newsletter content, and any additional information such as multimedia items is sent to the application server.
 3. The server interprets the information, recognizes that the request is for a new newsletter, generates the new record in the database, and performs background tasks (updating the newsletter counter, sending notification emails, etc).
 4. Next, the server sends a response back to the client. This does not necessarily mean that the newsletter was posted. The response could be that there was an error posting, or something along those lines. However, in this example we will say that the post went through properly, so the server sends a 201 response code and tells the browser which page to go to and render.
 5. Lastly, the browser receives the server information and shows you a message saying that your newsletter was successfully posted.
-

RESTful Conventions in Flask

Flask is lightweight, well-documented, and has a large community online (which gives you a wide population of people to provide help as you implement new tools in your Flask applications). Additionally, Flask provides native support for all HTTP request methods and makes it very easy to define routes- with these features, we can say that Flask natively supports REST conventions.

Let's take a look at how this would work for our newsletter application. RESTful routes fall into four familiar categories: Create, Read, Update, and Delete. These will roughly translate to five routes:

Category	Action
Create	Create the new newsletter instance.
Retrieve All (Index)	Display a list of all newsletters.
Retrieve One	Display an individual newsletter.
Update	Update the newsletter instance.
Delete	Delete an existing newsletter instance

HTTP Request Methods and RESTful Routes

To implement each of the above actions, we combine an HTTP request method (or *HTTP verb*) such as **GET** or **POST** with a route. Flask then maps each method/route combination to the appropriate view in the Flask application. The table below shows the HTTP request method and route we could use for this RESTful newsletter app:

HTTP Request Method	Route	Description
GET	/newsletters	Show all newsletters.
POST	/newsletters	Create a new newsletter.
GET	/newsletters/<int:id>	Show a specific newsletter.
PATCH or PUT	/newsletters/<int:id>	Update a specific newsletter.

HTTP Request Method	Route	Description
DELETE	<code>/newsletters/<int:id></code>	Delete a specific newsletter.

Note that even though we have five separate actions, we only have two routes defined in our application: `/newsletters` and `/newsletters/<int:id>`.

Flask does a great job of integrating RESTful routes into its system. If you can understand routes in Flask, you can understand REST in general.

► *How do RESTful routes benefit us as developers?*

Review of HTTP Request Methods

So what do `GET`, `POST`, et al. represent? These HTTP verbs give each HTTP request unique behavior. Below is an explanation of each verb:

- **GET:** The GET method retrieves whatever information is identified by the Request URI. This means if you go to `/newsletters`, you will get all of the newsletter posts that the application has.
- **POST:** The POST method is used to send data enclosed in the request to the server. The server is expected to use this data to create some new resource.
- **PATCH/PUT:** The PATCH and PUT methods are both used to update existing resources. Sending either a `PATCH` or `PUT` request to `/newsletters/1` will update the post with an `id` of 1. `PUT` is used when we want to replace an entire resource. `PATCH` is used when we want to update a specific part of a resource. Check out this explanation of the [difference between PUT and PATCH](https://www.geeksforgeeks.org/difference-between-put-and-patch-request/)  (<https://www.geeksforgeeks.org/difference-between-put-and-patch-request/>).
- **DELETE:** The DELETE method requests that the server delete the resource identified by the Request URI. This means... that it deletes the record. It's nice and explicit.

Things to Keep in Mind throughout this Module

Below are a few keys to remember when thinking about REST:

- REST is an architectural design pattern, not a framework or code in itself. Many other web frameworks utilize RESTful design principles in some form or another. By using RESTful principles, Flask apps are able to have a clear and standardized naming structure for routes and actions.
- RESTful routes have a clear mapping between the URL resource and the corresponding actions carried out by the backend.
- There are five RESTful route options we will commonly use as API developers.

► *What are the five RESTful route options we've discussed?*

Resources

- [What RESTful Actually Means](https://codewords.recurse.com/issues/five/what-restful-actually-means) ↗ [_ \(https://codewords.recurse.com/issues/five/what-restful-actually-means\)](https://codewords.recurse.com/issues/five/what-restful-actually-means)
- [What is an API? - MuleSoft](https://www.mulesoft.com/resources/api/what-is-an-api) ↗ [_ \(https://www.mulesoft.com/resources/api/what-is-an-api\)](https://www.mulesoft.com/resources/api/what-is-an-api)
- [HTTP request methods - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods) ↗ [_ \(https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods\)](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods)